How much do language models memorize?

John X. Morris[1,3], **Chawin Sitawarin**[2], **Chuan Guo**[1], **Narine Kokhlikyan**[1], **G. Edward Suh**[3,4], **Alexander M. Rush**[3], **Kamalika Chaudhuri**[1], **Saeed Mahloujifar**[1]

[1]FAIR at Meta, [2]Google DeepMind, [3]Cornell University, [4]NVIDIA

We propose a new method for estimating how much a model "knows" about a datapoint and use it to measure the capacity of modern language models. We formally separate memorization into two components: *unintended memorization*, the information a model contains about a specific dataset, and *generalization*, the information a model contains about the true data-generation process. By eliminating generalization, we can compute the total memorization of a given model, which provides an estimate of model capacity: our measurements estimate that **models in the GPT family have an approximate capacity of 3.6 bits-per-parameter**. We train language models on datasets of increasing size and observe that models memorize until their capacity fills, at which point "grokking" begins, and unintended memorization decreases as models begin to generalize. We train hundreds of transformer language models ranging from $500K$ to $1.5B$ parameters and produce a series of scaling laws relating model capacity and data size to membership inference.

Date: June 19, 2025
Correspondence: Saeed Mahloujifar at saeedm@meta.com

Meta

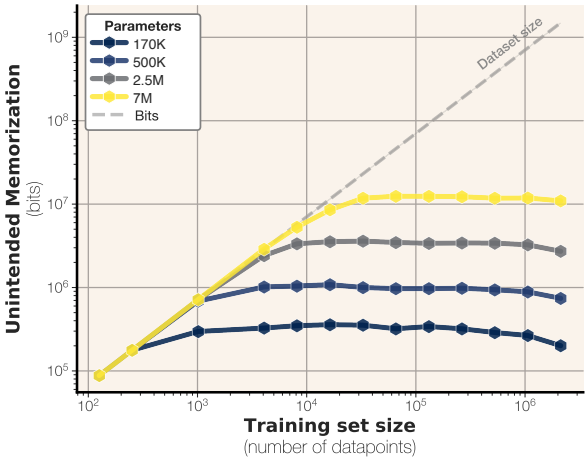

Figure 1 Unintended memorization of uniform random data (Section 3). Memorization plateaus at the empirical capacity limit of different-sized models from the GPT-family, approximately 3.6 bits-per-parameter.

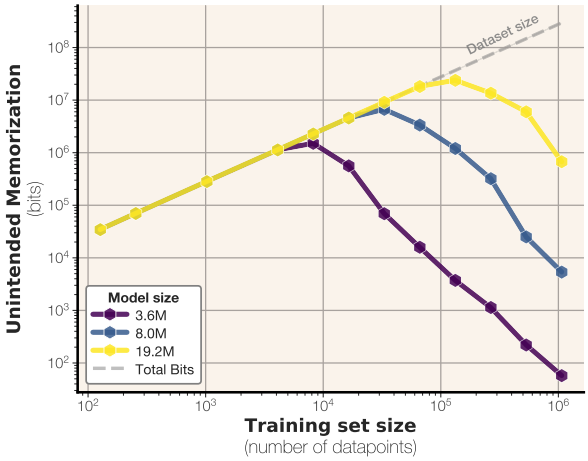

Figure 2 Unintended memorization of text across model and dataset sizes (Section 4). All quantities are calculated with respect to a large oracle model trained on the full data distribution.

1 Introduction

For the past several years, modern language models have been trained on increasingly large amounts of data, while parameter counts stay stagnant in the billions. For example, one recent state-of-the-art model (Dubey & et al, 2024) has 8 billion parameters (around $32GB$ on disk) but is trained on 15 trillion tokens (around $7TB$ on disk).

A long line of work (Carlini et al., 2019; Mireshghallah et al., 2022; Nasr et al., 2023; Zhang et al., 2023; Carlini et al., 2023b; Schwarzschild et al., 2024) questions whether such pretrained language models memorize their training data in a meaningful way. Most research approaches this problem either through the lens of

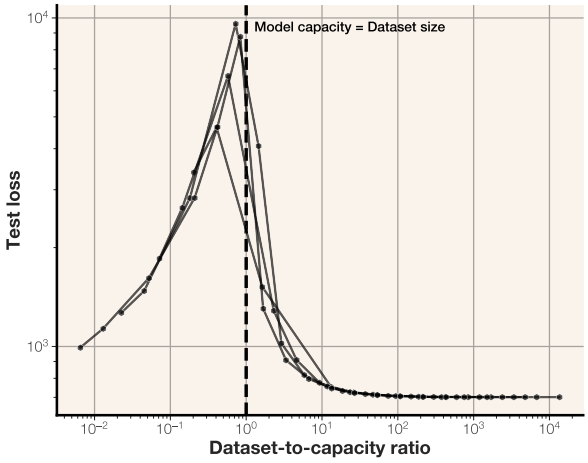

Figure 3 In our experiments on synthetic bitstrings, double descent occurs exactly when the dataset size begins to exceed the model's capacity, when unintended memorization is no longer beneficial for lowering the loss.

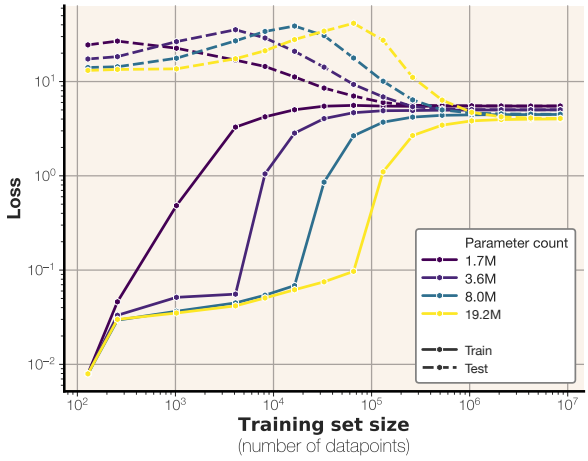

Figure 4 Train and test losses of different model and dataset sizes trained on text. Double descent occurs when dataset size exceeds model capacity.

extraction, aiming to recover full training datapoints from model weights, or membership inference, classifying whether a training point was present in the training data of a given model.

Studies of language model extraction argue that a datapoint is memorized if we can induce the model to generate it (Carlini et al., 2023b; Nasr et al., 2023; Schwarzschild et al., 2024). We argue that such generation does not necessarily serve as a proof of memorization. Language models can be coerced to output almost any string (Geiping et al., 2024); hence the fact that a model outputs something is not necessarily a sign of memorization. To address this issue, some researchers have suggested regularizing the input to the language model, such as by limiting its length (Schwarzschild et al., 2024) or matching it to the prefix (Carlini et al., 2023b) preceding the memorized sentence. However, none of these constraints allow us to distinguish whether a model outputs a string due to memorization or good generalization. For example, a language model prompted to add two numbers can output the answer without having seen the equation before.

To address this issue, we propose a definition of memorization that quantifies the extent to which a model retains information about a specific datapoint. Our approach leverages the concept of compression rate in bits: a model is considered to have memorized an input if the input can be compressed into a shorter encoding when the model is available. This framework draws inspiration from Kolmogorov information theory (Kolmogorov, 1963) and Shannon information theory (Shannon, 1948), but remains practical by estimating information content using model likelihoods. We address the fundamental challenge of distinguishing memorization from generalization (Prashanth et al., 2024) by decomposing memorization into two distinct components: *unintended memorization*, which captures the information the model retains about a specific dataset, and *generalization*, which represents the knowledge the model acquires about the underlying data-generating process. This separation is similar to the approach in (Brown et al., 2021), which defines memorization using conditional mutual information between the dataset and the trained model, conditioned on the true concept. However, our notion differs in that it enables this separation at the instance level using algorithmic definitions of information.

To understand our new quantities, we measure unintended memorization and generalization by training language models of varying capacity on datasets of different sizes. We first eliminate the question of generalization entirely by training on a dataset of random uniformly-sampled bitstrings. In this setting, we can exactly measure the amount of information contained about the data inside the model. This gives us a principled way to measure language model *capacity* when trained on uniform datasets of exact known information content. We find that GPT-style transformers can store between 3.5 and 4 bits of information in each model parameter, depending on model architecture and precision.

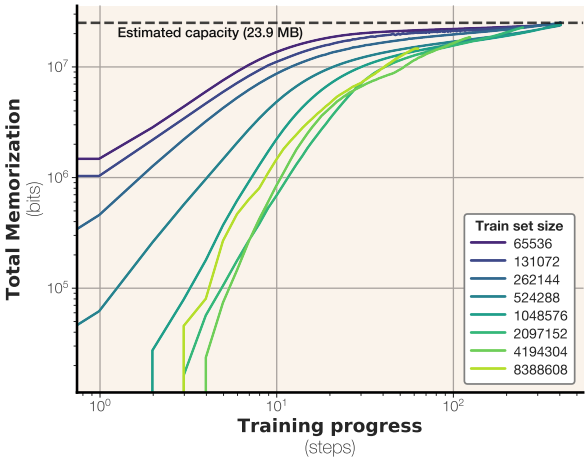

Figure 5 Bits memorized across training. This particular model is a GPT-style transformer with $6.86M$ parameters and a capacity of 23.9 MB.

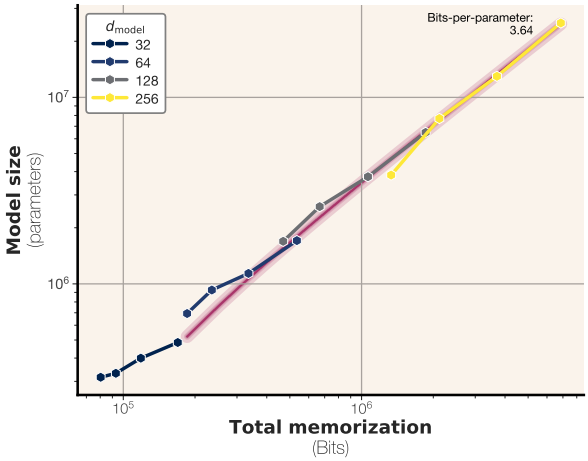

Figure 6 Capacity in bits-per-parameter for models trained on synthetic data. We estimate $\alpha = 3.64$ bits-per-parameter for GPT models trained in half precision.

We then repeat our experiments with real text, where generalization is possible and even beneficial for learning. On real text, language models memorize up to a certain capacity, at which point they substitute unintended memorization for generalization, and begin to learn general, reusable patterns as opposed to sample-level specifics. Our framework shows that double descent phenomenon begins to occur at this point, when the data size exceeds the model capacity in bits.

Finally, we use our results to predict a scaling law for membership inference performance based on model capacity and dataset size. We show that membership inference follows a clean relationship based on model capacity and dataset size: bigger models can memorize more samples, and making datasets bigger makes membership inference harder. Our scaling laws extrapolate to larger models, and predict most modern language models are trained on too much data to do reliable membership inference on the average datapoint.

2 Memorization, intended and unintended

When a model $\theta = L(x)$ is trained using a training algorithm L and a dataset $x \sim X$, some information is transferred from the sample x to the model θ. A key question in the memorization literature is determining how much of this stored information is intended versus unintended. In this work, we aim to provide a rigorous definition of memorization that satisfies certain properties:

1. *Separation from generalization.* Our notion of unintended memorization must be distinct from intended memorization, which we refer to as generalization. For example, consider a language model trained on the sample: *Q: What is* 2^{100}*? A: 1267650600228229401496703205376.* When assessing how much of this training sample is memorized, we must account for the fact that performing simple math operations is expected from a language model.

2. *Sample-level memorization.* We need to define memorization for realizations of random variables, not the random variables themselves. Specifically, we want to determine how much unintended memorization of a sample x occurs in a model θ.

3. *Independence from training algorithm.* Our definition should be independent of the training algorithm L and only a function of the final model θ and the sample x. This is crucial for language models, where we often only have access to the final model and target sample.

Previous works have attempted to define memorization for machine learning models. We aim to provide precise definitions of memorization that meet our criteria, and offer ways to measure it. See Appendix B for a broader discussion on definitions of memorization.

2.1 Warm-up: A statistical view of memorization

Notation. In this section, we use capital letters (e.g. X, Θ) to refer to random variables and lowercase letters to refer to instances of a random variable (e.g. $x \sim X$ and $\theta \sim \Theta$).

We rely on information theory, which has developed well-understood notions of information for random variables. For a random variable X, we use $H(X)$, the entropy of X, to define the amount of information present in X. For two distinct random variables X, Y, we can define $X \mid Y$ as the uncertainty left in X after fixing Y. With these definitions, we can now measure *mutual information* between X and Y by subtracting the leftover information from the total information: $I(X, Y) = H(X) - H(X \mid Y)$.

Now suppose we have a machine learning pipeline. We have a prior Θ on the underlying model that captures our dataset distribution X, and a learning algorithm L that maps samples from X to a trained model $\hat{\Theta}$. To understand how much information about X is stored in $\hat{\Theta}$, we can use the notion of mutual information:

$$\text{mem}(X, \hat{\Theta}) = I(X, \hat{\Theta}) = H(X) - H(X \mid \hat{\Theta}).$$

Note that this captures all the information about X that is stored in $\hat{\Theta}$. As we discussed, we need our notion of memorization to account for generalization as well. So when measuring unintended memorization, we are only interested in the information that is present in $X \mid \Theta$, which is the uncertainty left in X after fixing Θ. Hence, we can define **unintended memorization** as

$$\text{mem}_U(X, \hat{\Theta}, \Theta) = I([X \mid \Theta], \hat{\Theta}) = H(X \mid \Theta) - H(X \mid (\Theta, \hat{\Theta})).$$

and then the **generalization** (or intended memorization) must be

$$\text{mem}_I(X, \hat{\Theta}, \Theta) = \text{mem}(X, \hat{\Theta}) - \text{mem}_U(X, \hat{\Theta}, \Theta) = I(X, \hat{\Theta}) - I([X \mid \Theta], \hat{\Theta})$$

having defined our notions of intended and unintended memorization we turn our attention to practically measuring them. Let us first state a proposition that enables measurement of unintended memorization:

Proposition 1 (Super-additivity of Unintended Memorization). Assume $X = (X_1, \ldots, X_n)$ is a dataset of n i.i.d. samples. We have

$$\sum_{i \in [n]} \text{mem}_U(X_i, \hat{\Theta}, \Theta) \leq \text{mem}_U(X, \hat{\Theta}, \Theta) \leq H(\hat{\Theta}).$$

This proposition shows that to measure a lower bound on the unintended memorization on the dataset level, we can sum per-sample memorization. On the other hand, the entropy of the information content of the trained model itself servers as an upper bound on the unintended memorization. Another implication of this implies that unintended memorization should scale with the dataset size but cannot exceed the total capacity of the model.

We note that this statistical definition of memorization was first introduced by Brown et al. (2021), where they theoretically looked at the role of unintended memorization in enabling successful learning for certain tasks. However, this preliminary definition is unsuitable for us as we aim to practically measure instance-level memorization. In particular, since we observe only a single trained model and a single input sample, we are unable to define probabilities or conditional probabilities over samples conditioned on the model.

2.2 Measuring unintended memorization with Kolmogorov Complexity

Our definitions of memorization and generalization so far are defined using an “entropy-based" notion of information. This means our definitions can only be used for random variables. This brings big challenges in measuring memorization. All our variables in the definition of memorization are singletons. We have a single underlying model θ, we have a single dataset $x = (x_1, \ldots, x_n)$ and we have a single trained model $\hat{\theta}$[1]. It is impossible to measure the entropy (let alone conditional entropy) of the underlying variables using a single sample.

[1]Note the switch to lowercase variables because we are now working with instances, not random variables.